



AuxBMS V2

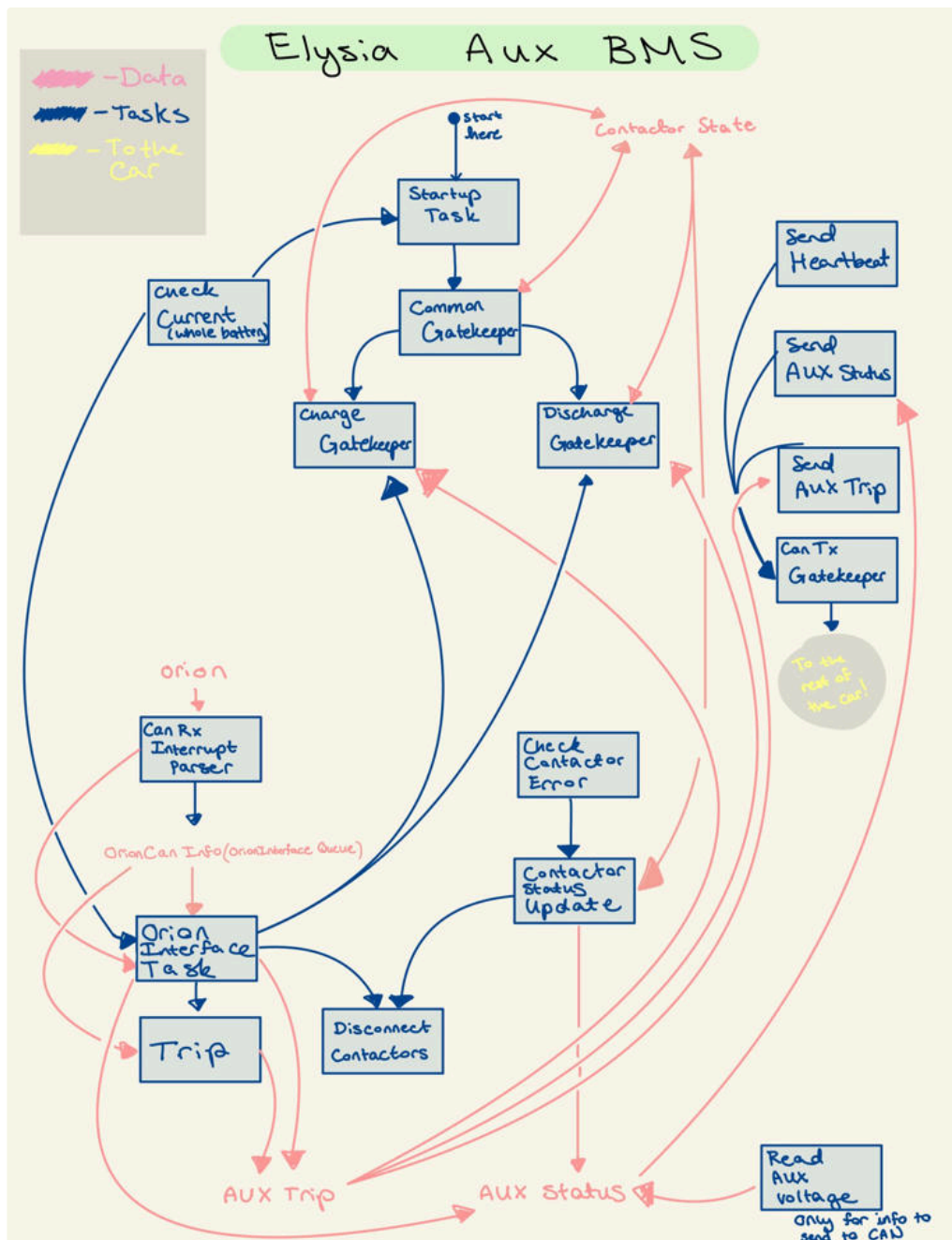
In the car, we have a big battery that powers the car. Along with it, a system called the 'Orion' controls it. It is a battery management system.

On our team, we decided to also create a custom battery management system (BMS) to have more control. We called this the 'AuxBMS'. We still use the Orion as it gathers data from the battery and does additional checks.

The embedded team is responsible for coding the microcontroller for our custom battery protection system. Then we can control the battery!

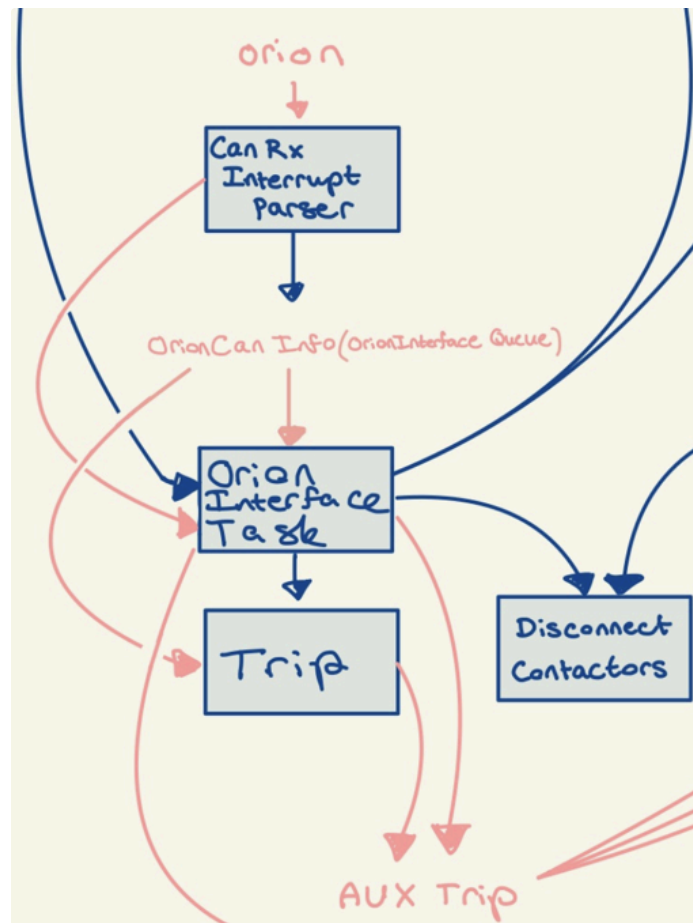
This is a safety-critical component. This system has to also send information to the rest of the car. The reason we send information to the rest of the car is because when the driver is driving, they cannot look into the battery nor do any testing, so we must inform them if anything goes awry. Now, let's take a deeper look into the AuxBMS.

Here is how Elysia, the old car's, AuxBMS looked like:



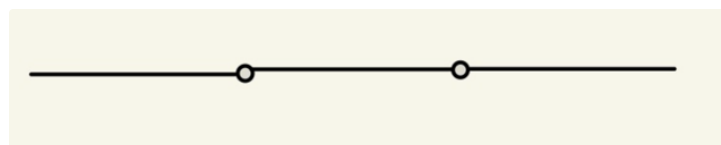
It's a little hectic but let's break it down.

First, let's look at the pink label 'Orion'.

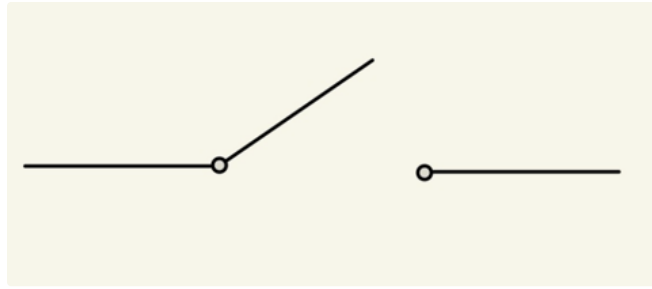


Here, the AuxBMS is getting data from the Orion we mentioned earlier. Then, we interrupt this data. 'RX' here means receive. After that, the data is put into a queue. This queue ensures that it's thread safe. That means the data can't become corrupted by one side writing to it and the other side reading it at the same time. Every object in that queue is a struct of *OrionCanInfo*. That's similar to saying it's an object of that type. Next, the *Orion Interface Task* grabs from the queue and then delegates what to do next. If needed, it will disconnect the contactors.

If the contactors are connected they look like this:



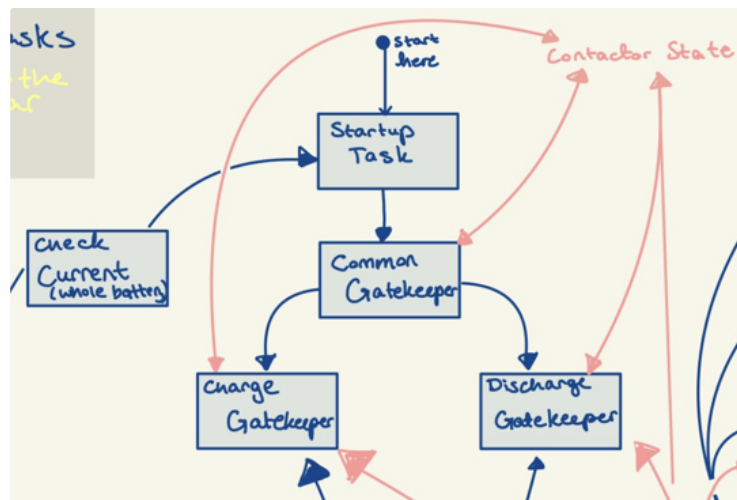
If they are disconnected they look like this:



Once they are disconnected the circuit will not work anymore and the battery will stop. We want this to happen in bad situations.

The *Orion Interface Task* will also send a message to the *Trip Task* if needed. Here, the *Trip Task* will update data of *Aux Trip*. Once this is updated, the gatekeepers will know that something is wrong and that will prevent them from opening any gates.

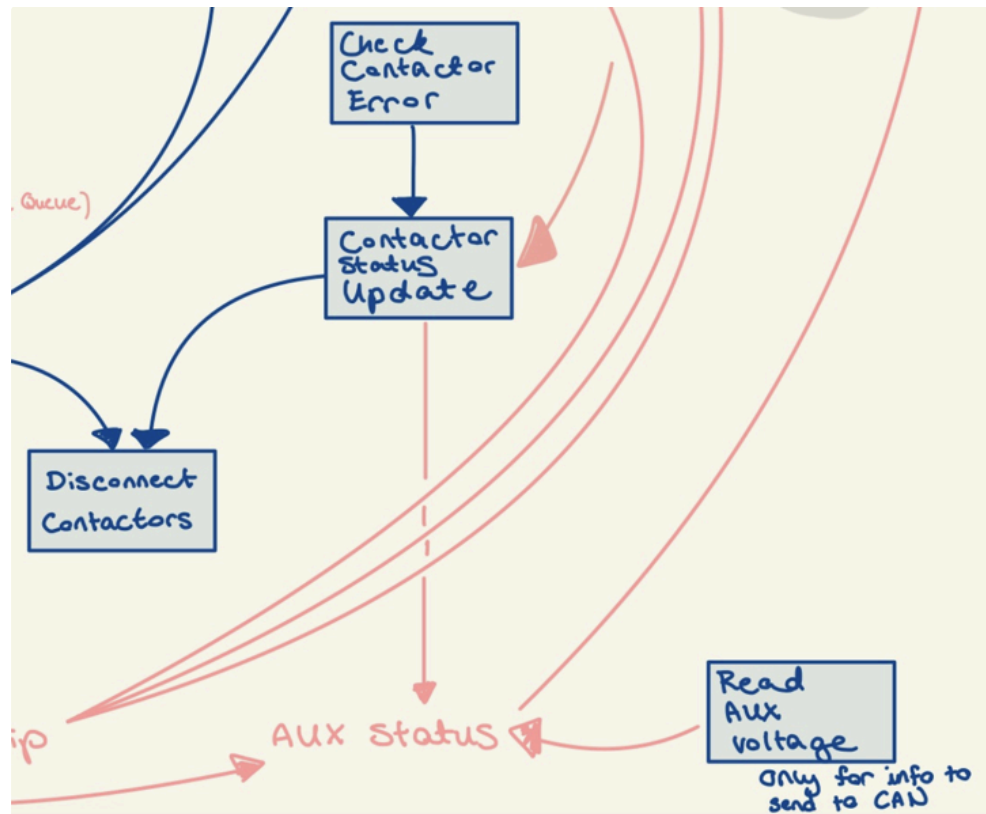
Speaking of gatekeepers, let's head up there and take a look at them:



Here, the *Startup Task* starts everything up. Once it begins, the *Check Current Task* checks the whole battery to see if anything suspicious is going on. Then, it sends this information to the *Orion Interface Task* and the *Startup Task*. If everything is good with the current, the *Startup Task* will proceed to the *Common Gatekeeper*. Here, data from the contactor state will inform the gatekeepers whether the contactors are closed, closing, or not. The gatekeeper in turn will update the contactor state.

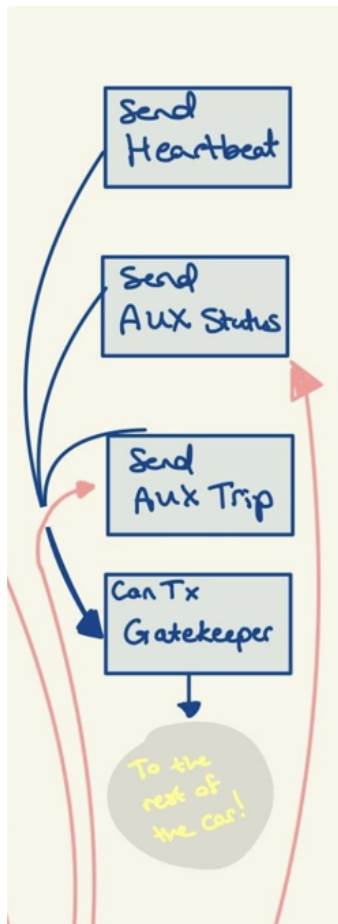
The common gatekeeper doesn't control which gatekeeper afterwards will happen. The common gatekeeper closes either way and then the *Orion Interface Task* informs whether the charge or discharge gatekeeper will close next. The pink arrow from the *Aux Trip* will open the gatekeepers if necessary.

We're almost done here, let's look at the bottom right of the diagram:



Here, the *Check Contactor Error* checks if there's any error and updates that status. This is then sent to the *Disconnect Contactors Task* and will disconnect the contactors if needed. It is also sent to the *Aux Status* and stored as data. The *Aux Voltage* sees the voltage and updates the *Aux Status* based off that. The last pink arrow is from the *Orion Interface Task* and that is updating the *Aux Status* with data of the battery and a lot of other data.

Finally, let's look at what's sent to the rest of the car.

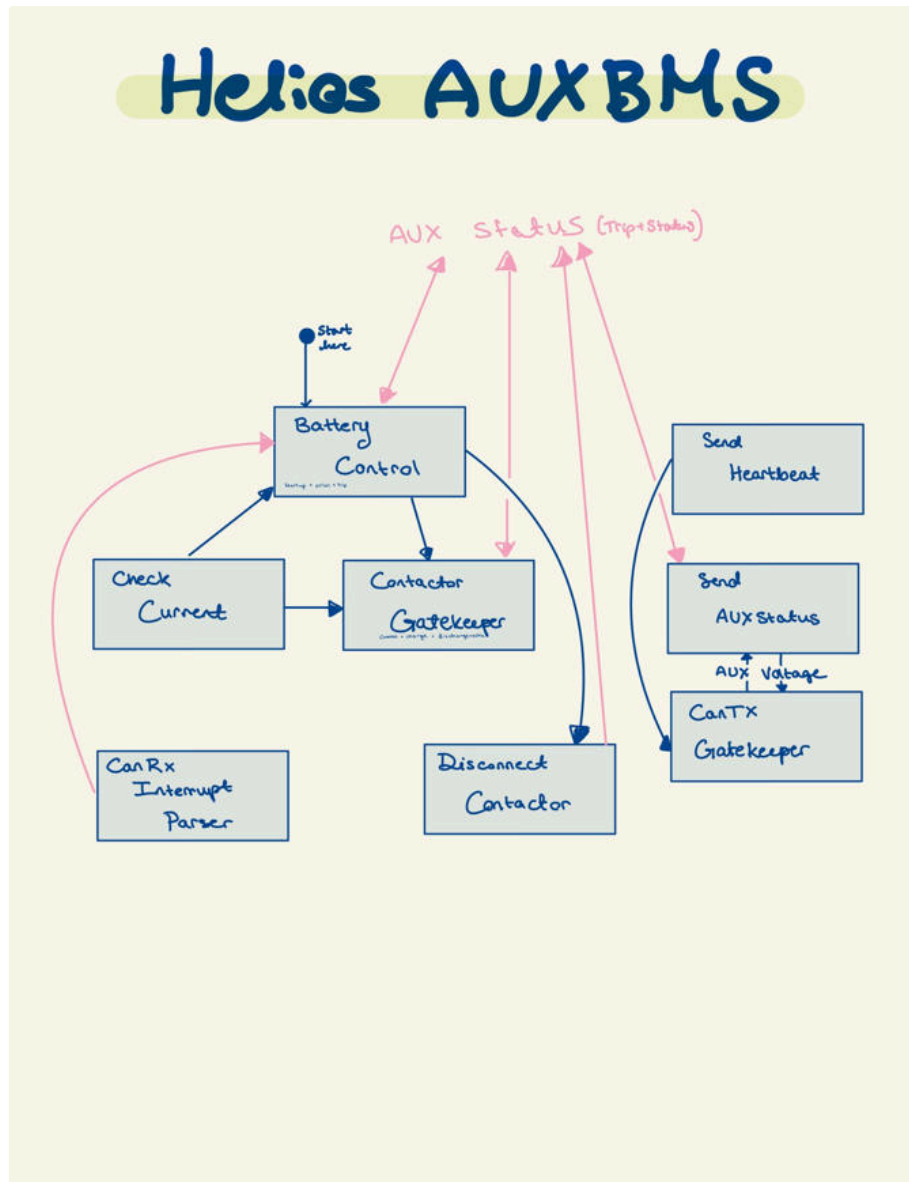


The *Heartbeat* says if the battery is alive and sends that status to the *CanTx Gatekeeper*. The *Aux Status* and *Aux Trip* send the data we defined earlier to the *CanTx Gatekeeper* too. 'Tx' means to send. Then the *CanTx Gatekeeper* sends all that data it got to the rest of the car.

So.

That was Elysia.

Now, let's look at the plan for Helios's AuxBMS.



elegance

Compared to Helios, it's been simplified a lot. First off, the *Startup Task* has been removed and created a *Battery Control Task*. This task combines the *Orion Interface Task* and the *Startup Task*. We also put the *Trip Task* into the *Battery Control Task*.

The *Contactor Gatekeeper* replaces the three separate *Gatekeeper* tasks from before and stores them all in one task. The plan is for this to store the common, charge, and discharge gatekeepers and additionally any others.

The *Contactor Gatekeeper* and *Battery Control Task* are getting information from *Aux Status* and updating it also. The *Disconnect Contactor* sends its status to the *Aux Status* too.

Compared to Elysia where we had multiple data points such as *Aux Trip*, *Aux Status*, *Contactor State*, we now only have one, the *Aux Status*, which stores all of this information instead.

The *Heartbeat* stays the same and so does the *Aux Status*. For the *Aux Voltage*, we get an ADC value from a pin on the board called the ADC pin and use that value to calculate the voltage. Then, we update that information along with the *Aux Status*. All that information gets sent to the *CanTX Gatekeeper* and gets sent to the car.

That's it. That's Helios' AuxBMS! Thanks for reading 🎉

Thank you Jessie and Marcelo for explaining everything to me :)